

Relazione - Versione 3: Rete con un Layer Nascosto

Dataset	Task	Architettura	Acc. test
Fashion-MNIST	10 classi	784 -> 64 -> 10	79.02%

Introduzione

La Versione 3 estende il modello lineare multiclasse aggiungendo un layer nascosto con 64 neuroni. L'obiettivo è introdurre non-linearità mantenendo l'implementazione compatta e completamente trasparente.

La rete è ancora costruita da zero con NumPy. Ora esegue un forward pass su due layer e propaga l'errore all'indietro attraverso entrambi.

Obiettivo della Versione 3

L'obiettivo principale è comprendere perché i layer nascosti sono utili e come la backpropagation distribuisce l'errore attraverso più insiemi di pesi e bias.

Dati e Preparazione

Il progetto utilizza il dataset Fashion-MNIST in formato CSV con 60.000 immagini etichettate di training e 10.000 immagini etichettate di test. Ogni immagine in scala di grigi 28 x 28 è rappresentata da 784 valori di pixel.

Vengono utilizzate tutte le 10 classi. I valori dei pixel sono normalizzati da 0-255 a 0-1 e le etichette intere vengono convertite in vettori one-hot per la cross-entropy multiclasse.

Architettura

Il flusso della Versione 3 è:

784 pixel -> 64 neuroni nascosti -> ReLU -> 10 punteggi di output -> softmax -> argmax

Layer Nascosto e ReLU

La prima matrice dei pesi ha forma 784 x 64 e mappa ogni immagine in 64 attivazioni nascoste. ReLU mantiene i valori positivi e porta a zero quelli negativi, permettendo alla rete di apprendere rappresentazioni non lineari.

Inizializzazione

Le due matrici dei pesi utilizzano una inizializzazione casuale in stile He. Un seed casuale fisso (42) rende l'esperimento riproducibile tra esecuzioni.

Softmax e Cross-Entropy

Il secondo layer mappa le 64 attivazioni nascoste in 10 punteggi di classe. La softmax trasforma questi punteggi in probabilità e la cross-entropy multiclasse misura l'errore di predizione.

Backpropagation

L'errore di output viene propagato all'indietro attraverso il secondo layer, la derivata della ReLU e poi il primo layer. Si ottengono così i gradienti per w_2 , b_2 , w_1 e b_1 .

Training

Il training usa gradient descent full-batch con learning rate 0,1 per 200 epoche fisse. Un numero fisso di epoche rende il tempo di esecuzione prevedibile e i risultati riproducibili.

Risultato del Training

Dopo 200 epoche, la loss finale di training è 0,5895 e l'accuratezza di training è 79,03%.

Valutazione

Il test segue la stessa normalizzazione e lo stesso forward pass a due layer. La classe con la probabilità softmax più alta viene selezionata con `argmax`. L'accuratezza sul test è 79,02%.

Risultati

Metrica	Valore
Training loss	0.5895
Training accuracy	79,03%
Test accuracy	79,02%

Cosa Insegna la Versione 3

La Versione 3 introduce la prima rete realmente non lineare del progetto. Il layer nascosto può trasformare i pixel originali prima della classificazione, mentre la backpropagation permette di addestrare insieme entrambi i layer.

La versione resta volutamente semplice: l'obiettivo è rendere visibili attivazioni nascoste, ReLU, inizializzazione dei parametri e backpropagation su due layer, non massimizzare l'accuratezza.

Prestazioni Attuali

Con questa precisa configurazione didattica, la rete raggiunge il 79,03% di accuratezza sul training e il 79,02% sul test. Aggiungere capacità rappresentativa non garantisce automaticamente risultati migliori: strategia di training e architettura devono evolvere insieme.

Limiti

La Versione 3 usa ancora gradient descent full-batch, learning rate fisso, un solo layer nascosto e nessuna regolarizzazione o optimizer avanzato. Il training è quindi più pesante del modello lineare, ma resta volutamente facile da ispezionare.

Conclusione

La Versione 3 aggiunge con successo un layer nascosto da 64 neuroni, attivazione ReLU, inizializzazione riproducibile e backpropagation attraverso due layer. La rete resta compatta e costruita da zero, creando la base architetturale per i miglioramenti successivi.